

파이썬 딕셔너리(Dictionary)

[1차시] 키와 값으로 데이터 저장하기

📖 핵심 개념 설명

리스트가 위치를 기준으로 값을 찾는다면 딕셔너리는 이름표인 키로 값을 찾습니다. 중괄호 안에 키와 값을 콜론으로 연결하여 저장합니다. 한 지역의 이름, 인구, 면적을 하나의 딕셔너리로 묶어 의미가 분명한 자료를 만듭니다.

📄 전체 실습 코드

```
district = {  
    "name": "한솔동",  
    "population": 18000,  
    "area": 2.8  
}  
  
print("전체 지역 정보:", district)  
print("지역 이름:", district["name"])  
print("인구:", district["population"])  
print("정보 항목 수:", len(district))
```

🔍 코드 훑아보기

- **키: 값** : name은 키이고 한솔동은 값입니다. 문자열 키는 따옴표로 표시합니다.
- **district["population"]** : 인덱스 대신 키를 사용해 인구 18000을 가져옵니다.
- **len(district)** : 키와 값의 쌍을 세므로 결과는 3입니다. 같은 키를 중복 작성하면 뒤의 값이 앞의 값을 덮어씁니다.

[2차시] 안전한 조회와 키 존재 확인

📖 핵심 개념 설명

자료마다 모든 항목이 들어 있는 것은 아닙니다. 존재하지 않는 키를 대괄호로 조회하면 `KeyError`가 발생합니다. `get`은 키가 없을 때 사용할 기본값을 지정할 수 있고 `in`은 키의 존재 여부를 확인합니다. 누락 항목을 안전하게 처리하는 조회 코드를 작성합니다.

📄 전체 실습 코드

```
district = {"name": "한솔동", "population": 18000}

print("지역:", district.get("name"))
print("면적:", district.get("area", "미조사"))
print("별도 기본값이 없을 때:", district.get("school"))

if "population" in district:
    print("인구 자료가 있습니다.")

print("한솔동이 키인가요?", "한솔동" in district)
print("한솔동이 값인가요?", "한솔동" in district.values())
```

🔍 코드 튜아보기

- `get(키, 기본값)` : `area`가 없으므로 미조사를 반환합니다. 기본값을 생략한 없는 키의 결과는 `None`입니다.
- `in district` : 값이 아니라 키를 검사합니다. `population`은 키이므로 조건이 참입니다.
- `values()` : 딕셔너리에 담긴 값들을 살펴봅니다. 한솔동이 값으로 있는지 확인하면 `True`가 나옵니다.

[3차시] 항목 추가와 값 수정

📖 핵심 개념 설명

딕셔너리의 키에 값을 대입하면 새 키는 추가되고 기존 키는 수정됩니다. 여러 항목을 한 번에 반영하려면 update를 사용합니다. 지역 자료에 조사 연도와 학교 수를 추가하고 인구를 수정하며 변경 규칙을 익힙니다.

📄 전체 실습 코드

```
district = {"name": "한솔동", "population": 18000}

district["year"] = 2025
district["population"] = 18500
print("추가와 수정:", district)

district.update({"population": 19000, "schools": 4})
print("여러 항목 반영:", district)
print("최종 인구:", district["population"])
print("전체 항목 수:", len(district))
```

🔍 코드 훑아보기

- **새 키에 대입** : year는 없던 키이므로 새 항목이 생깁니다. 키의 순서는 삽입 순서를 따릅니다.
- **기존 키에 대입** : population의 값이 바뀔 뿐 같은 이름의 항목이 추가되지는 않습니다.
- **update** : 기존 population은 19000으로 바꾸고 schools는 새로 추가합니다. 최종 항목 수는 4입니다.

[4차시] 항목 삭제와 기본값 등록

📖 핵심 개념 설명

불필요한 항목을 없앨 때 pop은 삭제한 값을 돌려주고 del은 지정한 항목을 삭제합니다. 반대로 setdefault는 키가 없을 때만 기본값을 등록합니다. 조회만 하는 get과 달리 덕서너리 자체를 바꿀 수 있다는 점을 구분합니다.

📄 전체 실습 코드

```
record = {"brand": "세종우유", "score": 85, "memo": "임시"}

removed = record.pop("memo", "없음")
print("삭제한 메모:", removed)
print("없는 항목 삭제:", record.pop("phone", "없음"))

record.setdefault("responses", 0)
record.setdefault("score", 0)
del record["brand"]

print("정리한 자료:", record)
```

🔍 코드 훑아보기

- **pop(키, 기본값)** : memo를 삭제하면서 임시를 반환합니다. 없는 phone은 기본값 없음을 반환하므로 오류가 나지 않습니다.
- **setdefault** : responses가 없어서 0을 저장합니다. score는 이미 있으므로 85를 유지합니다.
- **del record["brand"]** : brand 항목을 삭제합니다. 최종 자료는 score 85와 responses 0이며, 없는 키를 del로 삭제하면 오류가 납니다.

[5차시] keys·values·items와 반복문

📖 핵심 개념 설명

여러 지역을 한 번에 살펴보려면 딕셔너리의 키와 값을 순회해야 합니다. keys는 키, values는 값, items는 키와 값의 쌍을 제공합니다. 지역별 인구를 출력하고 전체 인구를 계산하며 반복문과 딕셔너리를 연결합니다.

📄 전체 실습 코드

```
populations = {"한솔동": 18000, "보람동": 32000, "새롬동": 25000}

print("지역 목록:", list(populations.keys()))
print("인구 목록:", list(populations.values()))

total = 0
for name, population in populations.items():
    print(name, ":", population, "명")
    total = total + population

print("전체 인구:", total)
print("내장 함수 합계:", sum(populations.values()))
```

🔍 코드 뜯아보기

- **keys와 values** : 키 또는 값만 골라 볼 수 있습니다. list로 변환하면 리스트 형태로 출력됩니다.
- **for name, population in ...items()** : 각 쌍의 키를 name에, 값을 population에 나누어 저장합니다.
- **합계 계산** : 세 지역의 합계는 75,000입니다. 딕셔너리를 직접 반복하면 기본적으로 키가 나온다는 점에 주의합니다.

[6차시] get을 활용한 빈도 집계

📖 핵심 개념 설명

같은 응답이 몇 번 나왔는지 집계할 때 딕셔너리가 유용합니다. 응답 내용을 키로 삼고 횟수를 값으로 저장합니다. 처음 나온 응답은 0에서 시작하고 이미 나온 응답은 기존 횟수에 1을 더하는 방식으로 선호 브랜드를 집계합니다.

📄 전체 실습 코드

```
responses = ["세종우유", "한솔제과", "세종우유",
             "세종푸드", "한솔제과", "세종우유"]
counts = {}

for brand in responses:
    counts[brand] = counts.get(brand, 0) + 1

print("브랜드별 응답 수:", counts)
for brand, count in counts.items():
    ratio = count / len(responses) * 100
    print(brand, count, "명", round(ratio, 1), "%")
```

🔍 코드 훑아보기

- 빈 딕셔너리 {}: 집계를 시작하기 전에는 어떤 브랜드가 나올지 몰라도 됩니다. 발견한 브랜드를 차례로 추가합니다.
- `get(brand, 0) + 1`: 처음 나온 브랜드는 `0 + 1`, 다시 나온 브랜드는 기존 횟수 + 1이 저장됩니다.
- 빈도와 비율: 세종우유 3명, 한솔제과 2명, 세종푸드 1명입니다. 전체 응답 수 6으로 나누어 비율을 구합니다.

[7차시] 중첩 딕셔너리로 지역 정보 관리

📖 핵심 개념 설명

지역마다 인구와 면적을 함께 저장하려면 값 자리에 또 다른 딕셔너리를 넣을 수 있습니다. 바깥쪽 키는 지역을, 안쪽 키는 정보 항목을 구분합니다. 계층 구조를 따라 값을 조회하고 인구 밀도를 계산해 봅니다.

📄 전체 실습 코드

```
districts = {
    "한솔동": {"population": 18000, "area": 2.8},
    "보람동": {"population": 32000, "area": 3.5}
}

print("한솔동 인구:", districts["한솔동"]["population"])
districts["한솔동"]["population"] = 18500

for name, info in districts.items():
    density = info["population"] / info["area"]
    print(name, "인구 밀도:", round(density, 1), "명/km2")
```

🔍 코드 훑아보기

- **키를 두 번 사용**: `districts["한솔동"]`은 안쪽 딕셔너리이며 여기에 `["population"]`을 붙여 숫자를 얻습니다.
- **안쪽 값 수정**: 한솔동의 인구만 18500으로 바꿉니다. 보람동의 자료에는 영향을 주지 않습니다.
- **밀도 계산**: 면적이 양수라는 가정에서 인구를 면적으로 나눕니다. 이 자료는 수업용 가상 자료이며 실제 지역 통계가 아닙니다.